

C++ Programming

Complete Reference Notes

From Fundamentals to the Standard Template Library

1. Introduction to C++

1.1 What is C++?

C++: a general-purpose, compiled, multi-paradigm programming language created by Bjarne Stroustrup at Bell Labs, beginning in 1979 as an extension of C originally called 'C with Classes'. It was renamed C++ in 1983.

C++ is often described as a 'superset' of C: almost all valid C code is also valid C++ code, but C++ adds object-oriented programming (classes, inheritance, polymorphism), generic programming (templates), exception handling, and a rich standard library (the STL). This makes C++ suitable for everything from operating systems and game engines to high-frequency trading systems and embedded firmware.

1.2 Key Characteristics of C++

- Multi-paradigm: supports procedural (like C), object-oriented, and generic programming styles in one language.
- Compiled language: source code is translated directly to machine code, producing fast executables, just like C.
- Statically typed: the type of every variable must be known at compile time (with some flexibility via auto and templates).
- Object-oriented: organizes code around classes and objects that bundle data with the functions that operate on it.
- Strong standard library: the Standard Template Library (STL) provides ready-made containers, algorithms, and iterators.
- Backward compatible with C: most C programs compile as C++ with little or no modification.
- Performance and control: like C, gives direct access to memory while adding higher-level abstractions on top.

1.3 C vs C++ — Quick Comparison

Aspect	C	C++
Paradigm	Procedural only	Procedural, object-oriented, and generic
Data + functions	Kept separate (structs hold data only)	Bundled together in classes
I/O	printf / scanf	cin / cout (also supports printf/scanf)
Memory management	malloc / free	new / delete (plus smart pointers)
Function overloading	Not supported	Supported
Standard library	Small (string.h, stdlib.h, etc.)	Much larger — includes the STL
File extension	.c	.cpp (also .cc, .cxx)

1.4 Structure of a C++ Program

Every C++ program follows a structure very similar to C, with a few additions such as namespaces and the iostream-based I/O style.

```
#include <iostream>    // Preprocessor directive (header inclusion)
using namespace std;  // Brings names like cout, cin into scope

const int MAX = 100;  // Global constant
void greet();         // Function prototype

int main()            // Entry point of every C++ program
{
    int localVar = 5;    // Local variable declaration
    cout << "Hello, World!" << endl; // Statement (stream output)
    greet();             // Function call
    return 0;           // Return statement (0 = success)
}

void greet()          // Function definition
{
    cout << "Welcome to C++ programming." << endl;
}
```

Explanation of each part

#include <iostream>: includes the input/output stream library, which provides cin, cout, and related I/O facilities.

using namespace std: lets you write cout instead of std::cout. The std namespace holds the entire C++ Standard Library to avoid naming collisions.

main() function: the mandatory entry point, exactly as in C. Execution always begins here.

cout / cin: the standard output and input stream objects, used with the << (insertion) and >> (extraction) operators.

1.5 Compiling and Running a C++ Program

Using the GNU Compiler Collection (g++), a typical workflow looks like this:

```
g++ program.cpp -o program    // Compile program.cpp into an executable named 'program'
g++ -std=c++17 program.cpp -o program // Compile using a specific C++ standard
./program                    // Run the executable (Linux/macOS)
program.exe                  // Run the executable (Windows)
```

Note: C++ has evolved through several standards: C++98, C++11, C++14, C++17, C++20, and C++23. C++11 was a landmark release introducing auto, smart pointers, lambdas, and range-based for loops. Always specify a standard explicitly (e.g. -std=c++17) on real projects to avoid relying on compiler-specific defaults.

2. Variables, Data Types, and Constants

2.1 Variables

Variable: a named location in memory used to store a value that can change during program execution, exactly as in C. C++ adds more flexible initialization syntax.

```
int age = 21;    // C-style initialization
```

```
float price (99.99f); // Constructor-style initialization
double rate {3.5};   // Brace (uniform) initialization — preferred in modern C++
int x, y, z = 0;      // multiple declarations
```

Note: Brace initialization `{}` is preferred in modern C++ because it prevents 'narrowing conversions' — e.g. `int x{3.9};` is a compile error, whereas `int x = 3.9;` silently truncates to 3.

2.2 Data Types

2.2.1 Primary (Basic) Data Types

Type	Typical Size	Description
int	4 bytes	Whole numbers (signed by default)
float	4 bytes	Single-precision floating-point number
double	8 bytes	Double-precision floating-point number
char	1 byte	A single character
bool	1 byte	Boolean value: true or false (new in C++, not in C)
void	0 bytes	Represents 'no value' — for functions returning nothing
wchar_t	2 or 4 bytes	Wide character, for extended character sets

Note: Use the `sizeof` operator to check the exact size on your system, e.g. `cout << sizeof(int);`. Sizes remain compiler/platform dependent, exactly as in C.

2.2.2 The bool Type

bool: a true C++ addition not present in C (which historically used `int`, with 0 = false and non-zero = true). `bool` variables hold only the literal values `true` or `false`, which are internally represented as 1 and 0.

```
bool isValid = true;
bool isEmpty = false;
cout << isValid << endl; // prints 1 (bool is printed as 0/1 by default)
```

2.2.3 Derived and User-Defined Data Types (overview — detailed later)

- Arrays — collection of elements of the same type (same as C).
- Pointers — variables that store memory addresses (same as C, with stricter type rules).
- References — an alias for an existing variable; a C++-only feature.
- Classes — the core of C++'s object-oriented system; group data AND functions under one name.
- Structures (struct) — in C++, structs can also contain functions, unlike in C.
- Enumerations (enum, enum class) — user-defined sets of named integer constants.

2.3 Constants

Constant: a value that cannot be changed during program execution.

```
const float PI = 3.14159; // 'const' qualifier — compiler enforced, preferred in C++
#define MAX_SIZE 100      // macro constant — inherited from C, used less in modern C++
constexpr int SIZE = 50;  // evaluated at COMPILE TIME — a C++11 addition
```

constexpr: tells the compiler the value must be computable at compile time, enabling optimizations and compile-time checks that `const` alone cannot guarantee. Prefer `constexpr` over `#define` for constants in modern C++.

2.4 Type Conversion

Implicit type conversion: automatic conversion performed by the compiler, following the same promotion rules as C.

Explicit type conversion (casting): C++ offers C-style casts as well as safer, more descriptive cast operators.

Cast	Purpose
<code>static_cast<T>(x)</code>	General-purpose, compile-time-checked conversion (e.g. int to float). The preferred everyday cast.
<code>dynamic_cast<T>(x)</code>	Safely converts pointers/references within a class hierarchy at runtime; returns <code>nullptr</code> on failure (requires polymorphism).
<code>const_cast<T>(x)</code>	Adds or removes the <code>const</code> qualifier from a variable.
<code>reinterpret_cast<T>(x)</code>	Reinterprets the bit pattern of one type as another; low-level, used sparingly.

```
int a = 7, b = 2;
float result = static_cast<float>(a) / b; // result = 3.5, the modern C++ way
float old_style = (float)a / b;          // C-style cast — still valid, but discouraged
```

2.5 Input and Output

C++ introduces stream-based I/O via `<iostream>`, an alternative to (and usually preferred over) C's `printf/scanf`.

Object / Operator	Purpose
<code>cout</code>	Standard output stream (console).
<code>cin</code>	Standard input stream (keyboard).
<code>cerr</code>	Standard error stream, typically unbuffered, for error messages.
<code><<</code>	Insertion operator — sends data INTO an output stream.
<code>>></code>	Extraction operator — reads data OUT OF an input stream.
<code>endl</code>	Inserts a newline and flushes the stream buffer.

```
#include <iostream>
using namespace std;

int main()
{
    int age;
    string name;

    cout << "Enter your name: ";
    cin >> name;          // reads a single word (no spaces)

    cout << "Enter your age: ";
    cin >> age;
```

Object / Operator	Purpose
	<pre>cout << "Hello " << name << ", you are " << age << " years old." << endl; return 0; }</pre>

Note: `cin >>` automatically skips leading whitespace and stops at the next whitespace, so it cannot read a full line containing spaces. Use `getline(cin, name)` to read an entire line, including spaces.

2.6 Namespaces

Namespace: a declarative region that provides a scope for identifiers (names of types, functions, variables), preventing naming collisions in large programs — a feature with no equivalent in C.

```
namespace MathUtils {
    int square(int x) { return x * x; }
}

int main() {
    int result = MathUtils::square(5); // :: is the scope resolution operator
    return 0;
}
```

Note: Writing `'using namespace std;'` at the top of every file is common in textbooks but discouraged in large/production codebases, since it imports the ENTIRE `std` namespace and increases the chance of name collisions. Many real projects prefer qualifying names (`std::cout`) or using targeted `'using std::cout;'` declarations.

3. Operators and Expressions

C++ inherits all of C's operators with identical behaviour, then adds a few of its own (most notably for memory management and references, covered in later sections).

3.1 Arithmetic, Relational, and Logical Operators

These behave identically to C:

Category	Operators
Arithmetic	<code>+ - * / %</code>
Relational	<code>== != > < >= <=</code>
Logical	<code>&& !</code> (short-circuit evaluated, same as C)
Bitwise	<code>& ^ ~ << >></code>
Assignment	<code>= += -= *= /= %= &= = ^= <<= >>=</code>
Increment/Decrement	<code>++ --</code> (pre and post forms, identical semantics to C)

3.2 The Conditional (Ternary) Operator

```
int a = 10, b = 20, max;
max = (a > b) ? a : b; // equivalent to: if (a > b) max = a; else max = b;
cout << max << endl; // 20
```

3.3 sizeof Operator

```
cout << sizeof(int) << endl;    // typically 4
cout << sizeof(double) << endl; // typically 8
```

3.4 Scope Resolution Operator (::)

Scope resolution operator (::): a C++-only operator used to qualify a name with the namespace, class, or global scope it belongs to, resolving ambiguity when an inner scope reuses an outer name.

```
int value = 10;    // global variable

int main() {
    int value = 20;    // local variable, same name
    cout << value << endl;    // 20 (local takes priority)
    cout << ::value << endl;    // 10 (:: forces global scope)
    return 0;
}
```

3.5 The new and delete Operators (preview)

C++ replaces C's malloc()/free() functions with the new and delete OPERATORS for dynamic memory. These are covered in depth in the Dynamic Memory section, but are introduced here as they are, technically, operators.

```
int *p = new int;    // allocate a single int on the heap
*p = 42;
delete p;            // release it
```

3.6 Operator Overloading (preview)

Operator overloading: a C++-only feature that lets you redefine what an operator like +, ==, or << means for objects of a user-defined class, so they can be used as naturally as built-in types. Covered fully in the Classes and Objects section.

Example preview: defining + for a Point class so that point1 + point2 adds their coordinates, rather than producing a compiler error.

3.7 Operator Precedence and Associativity

Precedence and associativity rules are the same as in C: multiplication/division bind tighter than addition/subtraction, and most binary operators associate left-to-right. When in doubt, use parentheses to make the intended order explicit.

4. Control Flow Statements

All of C's control flow statements work identically in C++. This section summarizes them and highlights the small C++-specific additions.

4.1 Decision-Making Statements

4.1.1 if / else / else-if

```
int marks = 82;
```

```

if (marks >= 90) {
    cout << "Grade A" << endl;
} else if (marks >= 75) {
    cout << "Grade B" << endl;
} else if (marks >= 40) {
    cout << "Grade C" << endl;
} else {
    cout << "Fail" << endl;
}

```

4.1.2 switch statement

Identical to C, including the need for break to avoid fall-through. C++17 additionally allows an init-statement inside if and switch.

```

switch (grade) {
    case 'A':
        cout << "Excellent" << endl;
        break;
    case 'B':
        cout << "Good" << endl;
        break;
    default:
        cout << "Invalid grade" << endl;
}

// C++17: init-statement scoped to the if
if (int status = getStatus(); status == 0) {
    cout << "Success" << endl;
}

```

4.2 Looping (Iteration) Statements

4.2.1 for loop (classic)

```

for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
// Output: 1 2 3 4 5

```

4.2.2 Range-based for loop (C++11 — not available in C)

Range-based for loop: a simplified loop syntax introduced in C++11 that iterates directly over the elements of an array, container, or any 'iterable' type, without manually managing an index.

```

int marks[5] = {90, 85, 78, 92, 88};
for (int m : marks) {           // 'for each m in marks'
    cout << m << " ";
}

for (auto &m : marks) {          // reference form — allows modifying elements
    m = m + 5;                   // adds 5 to every element in the original array
}

```

Note: Use 'for (auto &x : container)' when you need to modify elements in place, and 'for (const auto &x : container)' when you only need to read large/expensive-to-copy elements without modifying them.

4.2.3 while and do-while loops

Identical behaviour to C: while checks the condition before the body (entry-controlled); do-while checks it after (exit-controlled, guaranteeing at least one execution).

```
int i = 1;
while (i <= 5) {
    cout << i << " ";
    i++;
}

i = 1;
do {
    cout << i << " ";
    i++;
} while (i <= 5);
```

4.3 Jump Statements

Statement	Effect
break	Immediately terminates the nearest enclosing loop or switch statement.
continue	Skips the rest of the current iteration and moves to the next iteration.
goto	Transfers control to a labeled statement. Just as discouraged as in C.
return	Exits the current function, optionally returning a value to the caller.

5. Functions

Function: a self-contained, named block of code designed to perform a specific task, exactly as in C. C++ extends functions with overloading, default arguments, and reference parameters.

5.1 Function Declaration, Definition, and Call

The terminology (prototype, definition, call, parameters vs arguments) is identical to C.

```
#include <iostream>
using namespace std;

int add(int a, int b); // 1. Prototype (declaration)

int main() {
    int sum = add(3, 4); // 3. Function call
    cout << sum << endl; // Output: 7
    return 0;
}

int add(int a, int b) { // 2. Function definition
    return a + b;
}
```

5.2 Default Arguments (not available in C)

Default argument: a value automatically used for a parameter if the caller does not supply one. Default arguments must be specified from right to left — once one parameter has a default, all parameters after it must also have defaults.

```
void greet(string name, string greeting = "Hello") {
    cout << greeting << ", " << name << "!" << endl;
}

int main() {
    greet("Ayan");           // uses default: "Hello, Ayan!"
    greet("Ayan", "Welcome back"); // overrides default: "Welcome back, Ayan!"
    return 0;
}
```

5.3 Function Overloading (not available in C)

Function overloading: the ability to define multiple functions with the SAME name but different parameter lists (different number or types of parameters). The compiler selects the correct version based on the arguments used in the call.

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }

int main() {
    cout << add(2, 3) << endl;    // calls int version -> 5
    cout << add(2.5, 3.5) << endl; // calls double version -> 6.0
    cout << add(1, 2, 3) << endl;  // calls 3-argument version -> 6
    return 0;
}
```

Note: Overload resolution is based on the parameter list only, NOT the return type. Two functions differing only in return type (e.g. `int add()` vs `double add()`) are not valid overloads and will cause a compile error.

5.4 Parameter Passing Methods

5.4.1 Call by Value

A copy of the argument is passed; changes inside the function do not affect the original. Identical to C.

```
void modify(int x) {
    x = x + 10;    // only changes the local copy
}

int main() {
    int num = 5;
    modify(num);
    cout << num << endl; // still prints 5
    return 0;
}
```

5.4.2 Call by Pointer (same technique as C)

```
void modify(int *x) {
    *x = *x + 10;
}

int main() {
    int num = 5;
    modify(&num);
    cout << num << endl; // prints 15
    return 0;
}
```

5.4.3 Call by Reference (C++-only — no equivalent syntax in C)

Reference parameter: a parameter declared with &, which becomes an alias for the caller's actual argument. No separate dereferencing syntax (*) is needed inside the function — it behaves exactly like a normal variable, but changes affect the original.

```
void modify(int &x) {    // x is now an ALIAS for the caller's variable
    x = x + 10;          // directly changes the original — no * needed
}
int main() {
    int num = 5;
    modify(num);          // pass the variable directly, NOT its address
    cout << num << endl; // prints 15
    return 0;
}
```

Note: Call by reference is generally preferred over call by pointer in modern C++ when the function must always receive a valid object (references cannot be null and don't need null checks), reserving pointers for cases where 'no object' is a meaningful possibility.

5.5 Inline Functions

Inline function: a function for which the compiler is asked to substitute the function's code directly at the call site, avoiding the overhead of an actual function call, for very small/frequently-called functions. A C++ feature; the equivalent in C requires macros, which lack type safety.

```
inline int square(int x) {
    return x * x;
}
```

Note: 'inline' is only a suggestion to the compiler, which is free to ignore it. Modern compilers are generally better at deciding when to inline a function than the programmer is.

5.6 Recursion

Recursion works identically to C: a base case to stop, and a recursive case that moves toward it.

```
int factorial(int n) {
    if (n == 0) return 1;    // base case
    return n * factorial(n - 1); // recursive case
}
// factorial(4) = 4 * 3 * 2 * 1 = 24
```

5.7 Lambda Expressions (C++11 — not available in C)

Lambda expression: an anonymous, inline function that can be defined exactly where it is needed — commonly passed to algorithms, used as short callbacks, or stored in a variable. One of the most influential additions in C++11.

```
#include <iostream>
using namespace std;

int main() {
    auto square = [](int x) { return x * x; }; // lambda stored in a variable
    cout << square(5) << endl;                // 25

    int factor = 3;
    auto multiply = [factor](int x) { return x * factor; }; // captures 'factor' by value
    cout << multiply(4) << endl;                // 12
    return 0;
}
```

}	
Capture Syntax	Meaning
[]	Captures nothing from the surrounding scope.
[x]	Captures variable x BY VALUE (a copy).
[&x]	Captures variable x BY REFERENCE (can modify the original).
[=]	Captures ALL surrounding variables by value.
[&]	Captures ALL surrounding variables by reference.

5.8 Storage Classes

The same storage classes from C apply in C++ with identical meaning: auto (default, now mostly superseded by the type-inference 'auto' keyword's NEW meaning — see Section 15), static, extern, and register (largely obsolete, ignored by modern compilers).

Note: Caution: the keyword 'auto' has two very different meanings depending on the C++ standard. In pre-C++11 code (and in C), 'auto' was a rarely-used storage class specifier. From C++11 onward, 'auto' means 'deduce this variable's type automatically from its initializer' — by far its dominant modern use. See Section 15.1.

6. Arrays, Strings, and References

6.1 Arrays

C++ arrays behave identically to C arrays: same-type elements in contiguous memory, zero-indexed, with NO automatic bounds checking.

```
int marks[5] = {90, 85, 78, 92, 88};
cout << marks[0] << endl;    // 90
marks[2] = 80;                // modify an element

for (int i = 0; i < 5; i++) {
    cout << marks[i] << " ";
}
```

Note: As in C, accessing marks[10] on a 5-element array is undefined behaviour — it silently reads/writes adjacent memory rather than raising an error. This is just as dangerous in C++ as in C.

6.2 C-Style Strings vs the C++ string Class

C++ supports C's null-terminated char arrays for backward compatibility, but also provides a much safer, more convenient string class from the <string> header.

Aspect	C-style string (char[])	C++ string class
Header	string.h / cstring	<string>
Memory	Fixed-size, manually managed	Dynamically resized automatically
Concatenation	strcat() — risk of buffer overflow	+ operator — safe and automatic
Comparison	strcmp() — returns an int	== operator — returns a bool
Length	strlen()	.length() or .size()

Aspect	C-style string (char[])	C++ string class
Safety	No bounds checking	Bounds-checked with .at(); much safer overall

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string a = "Hello";
    string b = "World";

    string c = a + " " + b;    // concatenation with +, no buffer to manage
    cout << c << endl;        // Hello World
    cout << c.length() << endl; // 11

    if (a == "Hello") {        // direct comparison with ==
        cout << "Match!" << endl;
    }
    return 0;
}
```

Note: Prefer `std::string` over `char` arrays in almost all modern C++ code. C-style strings are still encountered when interfacing with C libraries or for very low-level/performance-critical work, but the `string` class eliminates an entire category of buffer-overflow bugs.

6.3 Multi-Dimensional Arrays

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
cout << matrix[1][2] << endl; // 6
```

6.4 References — A C++-Only Feature

Reference: an alias — an alternative name — for an already-existing variable. Once a reference is bound to a variable, it CANNOT be rebound to refer to a different variable; every operation on the reference acts directly on the original. C has no equivalent of this feature; C programmers simulate similar behaviour with pointers.

```
int num = 50;
int &ref = num;    // 'ref' is now another name for 'num' — must be initialized immediately

ref = 100;          // changes num too, since they are the same variable
cout << num << endl; // 100

num = 200;
cout << ref << endl; // 200 — they always stay in sync
```

6.4.1 References vs Pointers

Aspect	Pointer	Reference
Null value	Can be NULL / nullptr	Cannot be null — must refer to a real object
Rebinding	Can be reassigned to point elsewhere	Cannot be rebound after initialization

Aspect	Pointer	Reference
Syntax	Needs * to dereference	Used exactly like the original variable, no special syntax
Initialization	Can be declared uninitialized	Must be initialized at the point of declaration
Typical use	Optional/nullable data, dynamic structures, arrays of addresses	Function parameters, avoiding copies, operator overloading

6.4.2 References as Function Parameters (recap)

As shown in Section 5.4.3, references are the most common way C++ implements 'pass by reference', giving pointer-like efficiency (no copying) with ordinary-variable syntax.

6.4.3 const References

const reference: a reference qualified with `const`, commonly used as a function parameter to pass large objects (like a string or a custom class) WITHOUT copying them, while guaranteeing the function cannot modify the caller's original object.

```
void printName(const string &name) { // no copy made, and name cannot be modified here
    cout << name << endl;
    // name = "Changed"; // would be a compile error
}
```

Note: '`const string &name`' is the idiomatic way to pass strings and other non-trivial objects as function parameters in modern C++: it is as efficient as a pointer and as safe as pass-by-value.

7. Pointers

Pointers in C++ work identically to C at the fundamental level — a pointer stores the memory address of another variable. C++ adds the `nullptr` keyword and references as a frequently-preferred alternative, but pointers remain essential for dynamic data structures and low-level work.

7.1 Pointer Basics

```
int num = 50;
int *ptr = &num; // 'ptr' holds the ADDRESS of num

cout << num << endl; // 50 -- the value
cout << &num << endl; // e.g. 0x7ffee -- the address of num
cout << ptr << endl; // e.g. 0x7ffee -- same address, stored in ptr
cout << *ptr << endl; // 50 -- dereferencing: value AT this address
```

7.2 nullptr (C++11) vs NULL (inherited from C)

nullptr: a keyword introduced in C++11 representing a type-safe null pointer value, replacing the older C-style `NULL` macro (which is really just an integer 0 in disguise and can cause ambiguity with overloaded functions).

```
int *p = nullptr; // preferred modern C++ style
int *q = NULL;    // older C-style — still valid but less type-safe

if (p == nullptr) {
    cout << "p is not pointing anywhere yet" << endl;
}
```

7.3 Pointer Arithmetic

Identical to C: adding 1 to a pointer moves it forward by `sizeof(type)` bytes, not 1 byte, because elements of an array are laid out contiguously.

```
int arr[4] = {10, 20, 30, 40};
int *p = arr;      // points to arr[0]

cout << *p << endl;    // 10
p++;               // moves forward by sizeof(int) bytes
cout << *p << endl;    // 20
```

7.4 Pointers and Arrays

As in C, an array's name decays into a pointer to its first element when used in most expressions or passed to a function.

7.5 Pointers to Pointers and Void Pointers

These work exactly as in C. A `void*` can point to any data type but must be cast before being dereferenced.

7.6 Function Pointers

Function pointers work as in C, though in modern C++ they are increasingly replaced by lambdas and `std::function` (covered in Section 5.7 and the STL section) for clearer, more flexible code.

```
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

int main() {
    int (*operation)(int, int) = add;
    cout << operation(5, 3) << endl; // 8
    operation = sub;
    cout << operation(5, 3) << endl; // 2
    return 0;
}
```

7.7 Pointers and const

Declaration	Meaning
<code>const int *p</code>	p can point elsewhere, but the value it points to cannot be changed through p.
<code>int * const p</code>	p always points to the same address, but the value at that address can be changed.
<code>const int * const p</code>	Neither the address p holds, nor the value it points to, can change.

8. Dynamic Memory Management

C++ replaces C's `malloc()`/`calloc()`/`realloc()`/`free()` function-based approach with the `new` and `delete` OPERATORS, which are type-aware and integrate with constructors/destructors (see Section 9). C++ also introduces smart pointers, which manage memory automatically — arguably the single biggest practical improvement over C in this area.

8.1 Memory Layout of a C++ Program

Identical to C: Stack (local variables, function call frames), Heap (dynamic allocation), Data segment (global/static variables), and Code segment (compiled instructions).

8.2 new and delete

Operator	Purpose
<code>new T</code>	Allocates memory for a single object of type T on the heap and returns a typed pointer; calls the constructor if T is a class.
<code>new T[n]</code>	Allocates an array of n objects of type T on the heap.
<code>delete p</code>	Releases memory allocated with 'new T'; calls the destructor if T is a class.
<code>delete[] p</code>	Releases memory allocated with 'new T[n]'. Must match — never mix <code>new[]/delete</code> or <code>new/delete[]</code> .

```

int *p = new int;    // allocate a single int (uninitialized)
*p = 42;
delete p;           // release it
p = nullptr;        // good practice — avoid a dangling pointer

int *arr = new int[5]; // allocate an array of 5 ints
for (int i = 0; i < 5; i++) {
    arr[i] = i * i;
}
delete[] arr;        // release the ARRAY — note the []

```

8.2.1 new vs malloc — Quick Comparison

Aspect	malloc (C)	new (C++)
Type safety	Returns <code>void*</code> — requires manual casting	Returns a correctly-typed pointer automatically
Constructors	Does not call constructors	Automatically calls the object's constructor
Failure behaviour	Returns NULL on failure	Throws a <code>std::bad_alloc</code> exception on failure (by default)
Syntax	<code>malloc(n * sizeof(type))</code>	<code>new type</code> / <code>new type[n]</code>
Counterpart	<code>free()</code>	<code>delete</code> / <code>delete[]</code>

Note: Because `new` throws an exception rather than returning a null pointer on failure, code does not typically need to check for a null return after `new` the way it must after `malloc` — though catching `std::bad_alloc` is possible for programs that need to handle allocation failure gracefully.

8.3 Memory Leaks and Dangling Pointers

These are the exact same hazards as in C: forgetting `delete` leaks memory; using a pointer after `delete` (without setting it to `nullptr`) creates a dangling pointer.

```

void leak() {
    int *p = new int[100];
    // function ends WITHOUT delete[] p — the memory is now leaked
}

```


Note: This entire class of bug (manual new/delete mismatches, leaks, dangling pointers) is precisely the problem that smart pointers, introduced next, were designed to eliminate.

8.4 Smart Pointers (C++11) — A C++-Only Concept

Smart pointer: an object that wraps a raw pointer and automatically manages the lifetime of the memory it points to, calling delete automatically when the smart pointer goes out of scope. Smart pointers implement a broader idiom called RAI (Resource Acquisition Is Initialization): tie a resource's lifetime to an object's lifetime so it is released automatically and deterministically. Found in <memory>; there is no equivalent concept in C.

8.4.1 unique_ptr

unique_ptr: a smart pointer that exclusively owns the object it points to. It cannot be copied (only MOVED), guaranteeing only one unique_ptr ever owns a given resource at a time. This is the default, recommended smart pointer for sole ownership.

```
#include <memory>
using namespace std;

int main() {
    unique_ptr<int> p = make_unique<int>(42); // allocates and owns an int
    cout << *p << endl; // 42
    // No delete needed — automatically freed when p goes out of scope

    // unique_ptr<int> p2 = p; // COMPILE ERROR — cannot be copied
    unique_ptr<int> p2 = move(p); // OK — ownership is transferred (moved) to p2
    return 0;
} // p2's memory is automatically released here
```

8.4.2 shared_ptr

shared_ptr: a smart pointer that allows MULTIPLE owners of the same object via internal reference counting. The underlying memory is freed automatically only once the LAST shared_ptr pointing to it is destroyed.

```
#include <memory>
using namespace std;

int main() {
    shared_ptr<int> p1 = make_shared<int>(100);
    cout << p1.use_count() << endl; // 1
    {
        shared_ptr<int> p2 = p1; // shared ownership — copying IS allowed
        cout << p1.use_count() << endl; // 2
    } // p2 goes out of scope here, but the int is NOT freed yet
    cout << p1.use_count() << endl; // back to 1
    return 0;
} // memory is freed here, when the LAST shared_ptr (p1) is destroyed
```

8.4.3 weak_ptr

weak_ptr: a companion to shared_ptr that observes an object WITHOUT contributing to its reference count, used to break circular references between shared_ptrs that would otherwise prevent memory from ever being freed.

8.4.4 Smart Pointer Comparison

Type	Ownership	Copyable?	Typical Use
------	-----------	-----------	-------------

Type	Ownership	Copyable?	Typical Use
unique_ptr	Single, exclusive owner	No (movable only)	Default choice for most dynamically-allocated objects
shared_ptr	Multiple shared owners	Yes	Object needs to be accessed/owned by several parts of a program
weak_ptr	No ownership (observer only)	Yes	Breaking circular shared_ptr references

Note: In modern C++ (C++11 onward), raw pointers combined with manual new/delete are considered a last resort for owning dynamically allocated memory. Prefer smart pointers by default — they make the kind of memory leak shown in Section 8.3 effectively impossible, since the destructor runs automatically and deterministically.

9. Classes and Objects

Classes are the centerpiece of C++'s object-oriented programming model and have no equivalent in C. A class bundles data (member variables) together with the functions that operate on that data (member functions), modeling real-world entities far more directly than C's data-only structs.

9.1 Defining a Class

Class: a user-defined blueprint for creating objects, specifying what data (attributes) and behaviour (methods) every object of that type will have.

Object: a concrete instance of a class, with its own copy of the class's data members, occupying its own memory.

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    string name;    // data member (attribute)
    int roll;
    float marks;

    void display() { // member function (method)
        cout << name << " (Roll: " << roll << ") scored " << marks << endl;
    }
};

int main() {
    Student s1;      // s1 is an OBJECT of class Student
    s1.name = "Ayan"; // dot operator accesses members
    s1.roll = 101;
    s1.marks = 92.5;
    s1.display();    // Ayan (Roll: 101) scored 92.5
    return 0;
}
```

9.2 Access Specifiers

Access specifier: a keyword that controls which parts of a program can access a class's members, forming the basis of encapsulation. C has no equivalent — struct members in C are always fully open.

Specifier	Accessible From
public	Anywhere the object is visible — outside the class, by any code.
private	Only from within the class's own member functions (the default for class).
protected	From within the class AND any class that inherits from it (see Section 10).

Note: class members default to private; struct members default to public. This is the *ONLY* structural difference between class and struct in C++ — both can have constructors, member functions, access specifiers, and participate in inheritance.

9.3 Encapsulation

Encapsulation: the practice of bundling data together with the functions that operate on it, while hiding the internal representation from outside code (using private members) and exposing a controlled public interface (getters/setters). This protects an object's internal state from invalid or inconsistent modification.

```
class BankAccount {
private:
    double balance; // hidden from outside code

public:
    BankAccount() {
        balance = 0.0;
    }

    void deposit(double amount) {
        if (amount > 0) balance += amount; // validation happens here
    }

    double getBalance() { // controlled, read-only access
        return balance;
    }
};
```

Note: Without encapsulation, any code could set `balance = -5000` directly. By making `balance` private and only changing it through `deposit()/withdraw()`, the class can enforce its own rules (e.g. rejecting negative deposits) every time, everywhere.

9.4 Constructors

Constructor: a special member function, with the SAME NAME as the class and NO return type, that is automatically called whenever an object is created — used to initialize the object's data members.

9.4.1 Default Constructor

```
class Point {
public:
    int x, y;
    Point() { // default constructor — takes no arguments
        x = 0;
    }
};
```

```

        y = 0;
    }
};

int main() {
    Point p1; // default constructor called automatically — p1.x=0, p1.y=0
}

```

9.4.2 Parameterized Constructor

```

class Point {
public:
    int x, y;
    Point(int xVal, int yVal) { // parameterized constructor
        x = xVal;
        y = yVal;
    }
};

int main() {
    Point p1(10, 20); // calls the parameterized constructor
    cout << p1.x << ", " << p1.y << endl; // 10, 20
}

```

9.4.3 Constructor Initializer Lists (preferred modern style)

Member initializer list: a more efficient and idiomatic way to initialize data members, using a colon after the constructor's parameter list, BEFORE the constructor body runs. Required (not just preferred) for initializing const members and reference members.

```

class Point {
public:
    int x, y;
    Point(int xVal, int yVal) : x(xVal), y(yVal) { // initializer list
        // body can be empty — initialization already happened above
    }
};

```

9.4.4 Copy Constructor

Copy constructor: a constructor that initializes a new object as a copy of an existing object of the same class. The compiler generates one automatically if none is defined, but a custom one is needed when a class manages a resource (like dynamically allocated memory) that should not simply be copied byte-for-byte.

```

class Point {
public:
    int x, y;
    Point(int xVal, int yVal) : x(xVal), y(yVal) {}
    Point(const Point &other) : x(other.x), y(other.y) { // copy constructor
        cout << "Copy constructor called" << endl;
    }
};

int main() {
    Point p1(5, 10);
    Point p2 = p1; // invokes the copy constructor
}

```

9.5 Destructors

Destructor: a special member function, named `~ClassName`, with no return type and no parameters, automatically called when an object goes out of scope or is explicitly deleted. Used to release any resources (memory, file handles, etc.) the object acquired during its lifetime. C has no equivalent — programmers must remember to call cleanup functions manually.

```
class Logger {
public:
    Logger() {
        cout << "Logger created" << endl;
    }
    ~Logger() {           // destructor
        cout << "Logger destroyed" << endl;
    }
};

int main() {
    Logger log1;    // "Logger created" printed
    return 0;
}                  // "Logger destroyed" printed automatically here
```

Note: This automatic, guaranteed cleanup is the foundation of *RAII* (*Resource Acquisition Is Initialization*), the same idiom that makes smart pointers (Section 8.4) work: tie a resource to an object's lifetime, and let the destructor release it deterministically.

9.6 this Pointer

this: an implicit pointer available inside every non-static member function, pointing to the object on which the function was called. Most commonly used to disambiguate a member variable from a parameter of the same name, or to return the current object from a member function (e.g. for method chaining).

```
class Point {
public:
    int x, y;
    Point(int x, int y) {
        this->x = x;    // 'this->x' is the member; 'x' is the parameter
        this->y = y;
    }
};
```

9.7 Static Members

Static data member: a data member SHARED by all objects of a class, rather than each object having its own copy — similar in spirit to a static local variable in C, but scoped to the class. Must be defined (given storage) once outside the class.

Static member function: a member function that can be called without any object of the class, and can only access static data members (it has no 'this' pointer).

```
class Counter {
public:
    static int count;    // declaration only

    Counter() {
        count++;        // every new object increments the SHARED counter
    }

    static int getCount() { // static member function
        return count;
    }
}
```

```
};

int Counter::count = 0;      // definition — required exactly once, outside the class

int main() {
    Counter a, b, c;
    cout << Counter::getCount() << endl; // 3 — shared across all objects
}
```

9.8 Friend Functions and Classes

Friend function: a non-member function explicitly granted access to a class's private and protected members, declared inside the class with the 'friend' keyword. Used sparingly, typically for tightly-coupled operator overloads.

```
class Box {
private:
    int width;
public:
    Box(int w) : width(w) {}
    friend void printWidth(Box b); // grants access despite 'width' being private
};

void printWidth(Box b) {
    cout << b.width << endl; // allowed because of the friend declaration
}
```

9.9 Operator Overloading

Delivering on the preview from Section 3.6: operator overloading lets a class define what standard operators mean when applied to its objects.

```
class Point {
public:
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}

    Point operator+(const Point &other) { // overloading the + operator
        return Point(x + other.x, y + other.y);
    }

    void display() {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Point p1(2, 3), p2(4, 5);
    Point p3 = p1 + p2; // calls operator+ — now reads naturally
    p3.display();      // (6, 8)
    return 0;
}
```

Note: Common operators to overload include `+`, `-`, `==`, `<<`, and `=`. Overload the operators that genuinely make sense for the type (e.g. `+` for vectors/points/money), and avoid overloading an operator in a way that surprises readers about what it does.

9.10 Structs in C++ vs C

Unlike C structs (which can only hold data), C++ structs may have constructors, destructors, member functions, access specifiers, and can even participate in inheritance — the only difference from 'class' is the default access level (public for struct, private for class). In practice, C++ programmers typically reserve 'struct' for simple, mostly-data aggregates and 'class' for types with significant behaviour, by convention rather than necessity.

10. Inheritance

Inheritance: a mechanism that lets one class (the derived/child class) acquire the properties and behaviours of another class (the base/parent class), enabling code reuse and modeling 'is-a' relationships. Entirely absent from C, which has no concept of one struct 'extending' another.

10.1 Basic Inheritance Syntax

```
class Animal {           // base class
public:
    string name;
    Animal(string n) : name(n) {}
    void eat() {
        cout << name << " is eating." << endl;
    }
};

class Dog : public Animal { // derived class — 'public' inheritance
public:
    Dog(string n) : Animal(n) {} // calls the BASE class constructor
    void bark() {
        cout << name << " says Woof!" << endl;
    }
};

int main() {
    Dog d("Rex");
    d.eat(); // inherited from Animal: "Rex is eating."
    d.bark(); // Dog's own method: "Rex says Woof!"
    return 0;
}
```

10.2 Types of Inheritance

Type	Description
Single	One derived class inherits from exactly one base class (the example above).
Multiple	One derived class inherits from more than one base class simultaneously.
Multilevel	A class inherits from a derived class, which itself inherited from another base (a chain: A -> B -> C).
Hierarchical	Multiple derived classes inherit from the SAME single base class.
Hybrid	A combination of two or more of the above types in one design.
// Multiple inheritance — a feature C++ has that many later OOP languages (e.g. Java) deliberately omit class Flyable { public: void fly() { cout << "Flying..." << endl; } };	

Type	Description
<pre> class Swimmable { public: void swim() { cout << "Swimming..." << endl; } }; class Duck : public Flyable, public Swimmable { // inherits from BOTH }; int main() { Duck d; d.fly(); // from Flyable d.swim(); // from Swimmable } </pre>	

10.3 Access Control in Inheritance

The keyword used when inheriting (public, protected, or private) controls how the base class's members are exposed through the derived class.

Inheritance Type	public members become	protected members become
public	public in the derived class	protected in the derived class
protected	protected in the derived class	protected in the derived class
private	private in the derived class	private in the derived class

Note: public inheritance ('Dog : public Animal') is by far the most common, modeling a true 'is-a' relationship (a Dog IS an Animal). It is used in the vast majority of real-world inheritance hierarchies; protected and private inheritance are rare, specialized tools.

10.4 Constructor and Destructor Order

When an object of a derived class is created, the BASE class's constructor runs first, then the derived class's constructor. Destruction happens in the exact REVERSE order: derived class's destructor first, then the base class's.

```

class Base {
public:
    Base() { cout << "Base constructor" << endl; }
    ~Base() { cout << "Base destructor" << endl; }
};
class Derived : public Base {
public:
    Derived() { cout << "Derived constructor" << endl; }
    ~Derived() { cout << "Derived destructor" << endl; }
};

int main() {
    Derived d;
}
// Output:
// Base constructor
// Derived constructor
// Derived destructor
// Base destructor

```

10.5 Function Overriding

Function overriding: when a derived class defines a function with the SAME name, parameters, and signature as one in its base class, replacing the base class's version when called on a derived object.

```
class Animal {
public:
    void speak() {
        cout << "Some generic animal sound" << endl;
    }
};
class Cat : public Animal {
public:
    void speak() { // overrides Animal::speak()
        cout << "Meow!" << endl;
    }
};

int main() {
    Cat c;
    c.speak(); // "Meow!" — Cat's version is used
}
```

Note: This simple form of overriding works correctly only when calling through an object directly. When calling through a BASE class pointer or reference, you need 'virtual' functions — covered next, in Polymorphism — for the derived version to actually run.

11. Polymorphism

Polymorphism: from Greek for 'many forms' — the ability for the SAME function call or operator to behave differently depending on the actual type of object involved. C++ supports two kinds: compile-time (static) polymorphism via function/operator overloading, and runtime (dynamic) polymorphism via virtual functions.

11.1 Compile-Time Polymorphism (recap)

Already covered: function overloading (Section 5.3) and operator overloading (Section 9.9) are both resolved by the compiler at compile time, based on the static types involved.

11.2 Runtime Polymorphism — The Problem It Solves

Consider a base class pointer pointing to a derived class object. Without special handling, calling a function through that pointer always calls the BASE class's version — not the derived one — even if the object is actually a derived type:

```
class Animal {
public:
    void speak() { cout << "Generic animal sound" << endl; }
};
class Cat : public Animal {
public:
    void speak() { cout << "Meow!" << endl; }
};

int main() {
    Animal *a = new Cat(); // base class POINTER to a derived class OBJECT
    a->speak(); // prints "Generic animal sound" — NOT what we usually want!
    delete a;
}
```

11.3 virtual Functions — The Solution

Virtual function: a member function declared with the 'virtual' keyword in the base class, which tells the compiler to determine, AT RUNTIME, which version of the function to call based on the object's ACTUAL type — not the type of the pointer/reference used to call it. This mechanism (called dynamic dispatch) has no equivalent in C.

```
class Animal {
public:
    virtual void speak() { // 'virtual' enables runtime polymorphism
        cout << "Generic animal sound" << endl;
    }
};
class Cat : public Animal {
public:
    void speak() override { // 'override' (C++11) documents intent and is compiler-checked
        cout << "Meow!" << endl;
    }
};

int main() {
    Animal *a = new Cat();
    a->speak();           // now correctly prints "Meow!"
    delete a;
}
```

Note: The 'override' keyword (C++11) does not change behaviour, but causes a compile error if the function doesn't actually override anything in the base class (e.g. due to a typo in the signature) — catching a common, hard-to-spot class of bugs early.

11.4 Virtual Destructors

If a class is ever going to be deleted through a base class pointer, its destructor should be declared virtual — otherwise only the base class's destructor runs, and the derived class's resources may never be released.

```
class Base {
public:
    virtual ~Base() { // virtual destructor — always do this for base classes
        cout << "Base destroyed" << endl;
    }
};
class Derived : public Base {
public:
    ~Derived() {
        cout << "Derived destroyed" << endl;
    }
};

int main() {
    Base *b = new Derived();
    delete b; // with a virtual destructor, BOTH destructors run, in correct order
}
```

Note: Rule of thumb: if a class has ANY virtual function, give it a virtual destructor too. This is one of the most common C++ interview questions and a real source of subtle resource leaks in production code when forgotten.

11.5 Pure Virtual Functions and Abstract Classes

Pure virtual function: a virtual function with no implementation in the base class, declared by setting it '= 0'. Any class containing at least one pure virtual function becomes an abstract class, which CANNOT be instantiated directly — it exists only to be inherited from.

Abstract class: a class that defines an interface (a set of functions derived classes MUST implement) but provides no objects of its own. Conceptually similar to 'interfaces' in languages like Java, though C++ unifies the concept into the same 'class' keyword.

```
class Shape {                // abstract class
public:
    virtual double area() = 0; // pure virtual — no body, '= 0'
    virtual ~Shape() {}
};

class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() override {      // MUST implement area(), or Circle is also abstract
        return 3.14159 * radius * radius;
    }
};

int main() {
    // Shape s;           // COMPILE ERROR — cannot instantiate an abstract class
    Shape *s = new Circle(5);
    cout << s->area() << endl; // 78.53975 — runtime polymorphism in action
    delete s;
}
```

11.6 Polymorphism Summary

Concept	Resolved	Mechanism
Function overloading	Compile time	Different parameter lists, same name
Operator overloading	Compile time	Redefining operator behaviour for a class
Virtual functions	Runtime	vtable-based dynamic dispatch via base class pointer/reference
Pure virtual / abstract classes	Runtime	Forces derived classes to provide an implementation

12. Templates and Generic Programming

Template: a C++-only feature that lets you write a function or class that works with ANY data type, with the actual type filled in by the compiler at compile time based on how it is used. This is the foundation of 'generic programming' and underlies the entire Standard Template Library (STL). C has no direct equivalent — C programmers either duplicate code per type or use unsafe void* tricks and macros.

12.1 Function Templates

```
template <typename T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    cout << getMax(5, 10) << endl;    // T deduced as int  -> 10
    cout << getMax(3.5, 2.1) << endl;  // T deduced as double -> 3.5
    cout << getMax('a', 'z') << endl;  // T deduced as char  -> z
    return 0;
}
```

Note: Without templates, the same `getMax` logic would need to be written separately for `int`, `double`, `char`, and every other type — exactly the kind of repetition C programmers historically tolerated or worked around with *type-unsafe macros*.

12.2 Class Templates

```
template <typename T>
class Box {
private:
    T value;
public:
    Box(T v) : value(v) {}
    T getValue() { return value; }
};

int main() {
    Box<int> intBox(42);
    Box<string> strBox("Hello");

    cout << intBox.getValue() << endl; // 42
    cout << strBox.getValue() << endl; // Hello
    return 0;
}
```

12.3 Multiple Template Parameters

```
template <typename T1, typename T2>
class Pair {
public:
    T1 first;
    T2 second;
    Pair(T1 a, T2 b) : first(a), second(b) {}
};

int main() {
    Pair<string, int> p("Ayan", 22); // T1=string, T2=int
    cout << p.first << " is " << p.second << " years old." << endl;
}
```

12.4 Templates and the STL

Every container in the Standard Template Library — `vector`, `map`, `set`, and the rest, covered fully in Section 13 — is itself a class template. This is precisely why a single `'vector'` can hold ints, strings, or any custom class: `vector<T>` is generic, and you supply `T`.

13. Exception Handling

Exception handling: a structured mechanism for detecting and responding to runtime errors (like division by zero, invalid input, or failed memory allocation) without crashing the program or relying on manual error-code checking after every single function call. C has no built-in equivalent; C

programs typically rely on function return codes and the global `errno` variable, which are easy to forget to check.

13.1 try, throw, and catch

Keyword	Purpose
<code>try</code>	Wraps a block of code that might throw an exception.
<code>throw</code>	Signals that an exceptional/error condition has occurred, optionally carrying a value describing it.
<code>catch</code>	Handles an exception thrown inside a corresponding try block, matched by the type of value thrown.

```

#include <iostream>
using namespace std;

double divide(double a, double b) {
    if (b == 0) {
        throw runtime_error("Division by zero!"); // throw an exception
    }
    return a / b;
}

int main() {
    try {
        cout << divide(10, 0) << endl;
    }
    catch (const runtime_error &e) { // catches the specific exception type
        cout << "Error: " << e.what() << endl; // .what() returns the error message
    }
    cout << "Program continues normally." << endl;
    return 0;
}

```

Note: Unlike a crash, an uncaught condition handled with exceptions lets the program recover and CONTINUE running — 'Program continues normally.' still prints in the example above, after the error is handled.

13.2 Standard Exception Classes

The `<stdexcept>` header provides a hierarchy of standard exception types, all derived from `std::exception`, the common base class.

Exception Class	Typically Thrown For
<code>std::exception</code>	The base class for all standard exceptions; provides the <code>.what()</code> method.
<code>std::runtime_error</code>	Errors detectable only at runtime (e.g. invalid user input, failed operations).
<code>std::logic_error</code>	Errors that reflect a flaw in program logic (e.g. invalid arguments).
<code>std::out_of_range</code>	Accessing a container or array beyond its valid bounds.
<code>std::invalid_argument</code>	A function received an argument it cannot accept.
<code>std::bad_alloc</code>	Memory allocation with 'new' failed.

13.3 Catching Multiple Exception Types

```
try {
    // risky code
}
catch (const out_of_range &e) {
    cout << "Out of range: " << e.what() << endl;
}
catch (const exception &e) {    // catches ANY other standard exception
    cout << "Other error: " << e.what() << endl;
}
catch (...) {                  // catch-all — catches literally anything
    cout << "Unknown error occurred." << endl;
}
```

Note: *catch blocks are checked top to bottom, and the FIRST matching one runs — so always order catch blocks from most specific to most general, exactly like an if-else ladder.*

13.4 Custom Exception Classes

You can define your own exception types by inheriting from `std::exception`, allowing exceptions specific to your program's domain.

```
class InsufficientFundsException : public exception {
public:
    const char* what() const noexcept override {
        return "Insufficient funds for this withdrawal.";
    }
};

void withdraw(double balance, double amount) {
    if (amount > balance) {
        throw InsufficientFundsException();
    }
}

int main() {
    try {
        withdraw(100, 500);
    } catch (const exception &e) {
        cout << e.what() << endl;
    }
}
```

13.5 Stack Unwinding and RAII Interaction

Stack unwinding: the process that occurs when an exception is thrown: the runtime automatically calls the destructors of all local objects in scope, from the point of the throw back up to the matching catch block, before transferring control to the handler.

This is precisely why RAII (Section 8.4, Section 9.5) matters so much in exception-safe C++ code: because destructors run automatically during stack unwinding, a smart pointer or any RAII-managed resource is guaranteed to be cleaned up correctly even if an exception is thrown midway through a function — something that requires careful, manual cleanup code in C.

14. The Standard Template Library (STL)

Standard Template Library (STL): a powerful, template-based collection of ready-made containers, algorithms, and iterators, included with the C++ standard library. It is one of C++'s biggest practical advantages over C, which has no built-in data structures at all (Section 13 of the C document showed how a C programmer must hand-build even a basic linked list).

The STL has four main components: Containers (store collections of objects), Iterators (traverse containers generically), Algorithms (operate on ranges defined by iterators), and Function objects/lambda (customize algorithm behaviour).

14.1 Sequence Containers

14.1.1 *vector* — A Dynamic Array

vector: a dynamically-resizable array — the most commonly used STL container, and usually the right default choice for a sequential collection. Grows automatically as elements are added, unlike a C array's fixed size.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {10, 20, 30};
    nums.push_back(40);    // adds an element to the end — grows automatically
    nums.pop_back();       // removes the last element

    cout << nums[0] << endl;    // 10 — indexing works like a normal array
    cout << nums.size() << endl; // 3 — current number of elements

    for (int n : nums) {      // range-based for loop (Section 4.2.2)
        cout << n << " ";
    }
    return 0;
}
```

14.1.2 *list* — A Doubly Linked List

list: a doubly linked list, allowing fast insertion/removal anywhere in the sequence (constant time), but NOT supporting direct indexing like `vector[i]` — this is the STL's built-in equivalent of the linked list that Section 13.1 of the C document built by hand.

14.1.3 *deque* — Double-Ended Queue

deque: a sequence container that supports fast insertion and removal at BOTH the front and the back, unlike `vector` (which is only efficient at the back).

14.2 Associative and Unordered Containers

14.2.1 *map* and *unordered_map*

map: an associative container storing key-value pairs, automatically sorted by key, with no equivalent built into C or even basic Java/Python without imports. Lookups, insertions, and deletions run in logarithmic time, backed internally by a balanced binary search tree.

unordered_map: functions identically to `map` (key-value pairs) but offers average constant-time $O(1)$ operations using a hash table internally, at the cost of not keeping keys in sorted order.

```
#include <map>
using namespace std;

int main() {
    map<string, int> ages;
    ages["Ayan"] = 22;    // insert/update using [] like an array
    ages["Riya"] = 24;
}
```

```
cout << ages["Ayan"] << endl; // 22

for (auto &pair : ages) { // iterating a map gives key-value pairs
    cout << pair.first << ": " << pair.second << endl;
}
return 0;
}
```

14.2.2 set and unordered_set

set: a container that stores UNIQUE elements only, automatically kept in sorted order. Inserting a duplicate value has no effect.

```
#include <set>
using namespace std;

int main() {
    set<int> s = {5, 1, 3, 1, 5}; // duplicates are automatically discarded
    for (int x : s) cout << x << " "; // 1 3 5 — sorted, unique
    return 0;
}
```

14.3 Container Quick-Reference

Container	Stores	Key Property	Typical Use Case
vector	Sequence	Fast random access; grows at the back	General-purpose dynamic array (default choice)
list	Sequence	Fast insert/remove anywhere; no random access	Frequent mid-sequence insertions/removals
deque	Sequence	Fast insert/remove at both ends	Queues processed from both ends
map	Key-value pairs	Sorted by key; $O(\log n)$ operations	Lookups that also need sorted key order
unordered_map	Key-value pairs	Hash-based; $O(1)$ average operations	Fast lookups, order doesn't matter
set	Unique values	Sorted, no duplicates	Membership testing, removing duplicates
stack	Sequence (LIFO)	push/pop only at one end	Undo systems, expression evaluation, recursion-like logic
queue	Sequence (FIFO)	push at back, pop from front	Task scheduling, BFS traversal

14.4 stack and queue — Built-In, Unlike C

Recall that Section 13.2 of the C document described stack and queue only conceptually, since C provides no built-in versions — a programmer must implement them from scratch. The STL provides both directly, as container adapters built on top of deque by default.

```
#include <stack>
```



```
#include <queue>
using namespace std;

int main() {
    stack<int> st;
    st.push(10); st.push(20); st.push(30);
    cout << st.top() << endl; // 30 — most recently pushed (LIFO)
    st.pop();

    queue<int> q;
    q.push(10); q.push(20); q.push(30);
    cout << q.front() << endl; // 10 — first one pushed (FIFO)
    q.pop();
    return 0;
}
```

14.5 Iterators

Iterator: an object that points to an element within a container and can be advanced to 'step through' the container, providing a single, generic way for algorithms to traverse ANY container type without knowing its internal structure.

```
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {10, 20, 30};

    for (vector<int>::iterator it = nums.begin(); it != nums.end(); it++) {
        cout << *it << " "; // dereference the iterator to get the value
    }
    // begin() points to the first element; end() points ONE PAST the last element
    return 0;
}
```

Note: In modern C++, range-based for loops (Section 4.2.2) handle most simple iteration needs and are generally preferred for readability. Explicit iterators remain essential for algorithms that need to insert/erase elements, move backward, or work generically across container types.

14.6 Algorithms (<algorithm> header)

The STL provides dozens of generic algorithms that work on iterator ranges from ANY container — the same `sort()` works on a vector, a deque, or an array, because it operates purely through iterators rather than knowing the container's internal layout.

Algorithm	Purpose
<code>sort(begin, end)</code>	Sorts a range into ascending order by default.
<code>find(begin, end, val)</code>	Returns an iterator to the first occurrence of <code>val</code> , or <code>end()</code> if not found.
<code>max_element / min_element</code>	Returns an iterator to the largest/smallest element in a range.
<code>reverse(begin, end)</code>	Reverses the order of elements in a range.
<code>count(begin, end, val)</code>	Counts how many elements equal <code>val</code> .
<code>accumulate(begin, end, init)</code>	Sums (or otherwise combines) all elements in a range (from <code><numeric></code>).

Algorithm	Purpose
<pre> #include <algorithm> #include <vector> using namespace std; int main() { vector<int> nums = {40, 10, 30, 20}; sort(nums.begin(), nums.end()); // 10 20 30 40 auto it = find(nums.begin(), nums.end(), 30); if (it != nums.end()) { cout << "Found 30!" << endl; } auto biggest = max_element(nums.begin(), nums.end()); cout << *biggest << endl; // 40 return 0; } </pre>	

Note: Many STL algorithms accept a lambda (Section 5.7) as an optional argument to customize their behaviour — for example, `sort(v.begin(), v.end(), [](int a, int b){ return a > b; })` sorts in DESCENDING order instead of the default ascending.

14.7 Pair and Tuple

pair: a simple template (from <utility>) that bundles exactly two values, possibly of different types, accessed via .first and .second — used heavily as the element type when iterating a map (see Section 14.2.1).

tuple: a generalization of pair (from <tuple>) that can bundle any fixed number of values of different types, accessed with `std::get<index>(t)` or, in C++17 onward, structured bindings.

```

#include <tuple>
using namespace std;

int main() {
    tuple<string, int, double> student("Ayan", 22, 92.5);
    cout << get<0>(student) << endl;    // Ayan

    auto [name, age, marks] = student;  // C++17 structured bindings
    cout << name << " " << age << " " << marks << endl;
    return 0;
}

```

15. Modern C++ Features (C++11 and Later)

This section gathers the major language features introduced from C++11 onward that don't fit neatly under earlier topics, several of which were previewed in earlier sections and are expanded on here.

15.1 auto — Automatic Type Deduction

auto (C++11 meaning): tells the compiler to deduce a variable's type automatically from its initializer, rather than the programmer writing it out explicitly. Reduces verbosity, especially with long template types like iterators, while remaining fully statically typed — the type is fixed at compile time, just inferred rather than spelled out.

```

auto x = 10;           // deduced as int
auto y = 3.14;         // deduced as double
auto name = string("Ayan"); // deduced as string

vector<int> nums = {1, 2, 3};
for (auto it = nums.begin(); it != nums.end(); it++) { // avoids spelling out
    cout << *it << " ";                               // vector<int>::iterator
}

```

Note: *auto is a compile-time convenience, not a 'dynamic' or 'variant' type like in scripting languages — once deduced, the variable's type is fixed for its entire lifetime, exactly as if it had been written explicitly.*

15.2 Move Semantics and Rvalue References (C++11)

Rvalue reference (&&): a reference type that binds specifically to temporary objects ('rvalues') about to be destroyed, enabling 'move' operations that transfer ownership of resources (like heap-allocated memory) instead of expensively copying them.

Move constructor / move assignment: special member functions that 'steal' the internal resources of a temporary object (e.g. a pointer to heap data) rather than copying them, leaving the source object in a valid but empty state. The compiler automatically uses these whenever it can, for example when returning large objects from a function.

```

class Buffer {
private:
    int *data;
    size_t size;
public:
    Buffer(size_t s) : size(s) {
        data = new int[s];
    }

    // Move constructor — steals 'other's pointer instead of copying the array
    Buffer(Buffer &&other) noexcept : data(other.data), size(other.size) {
        other.data = nullptr; // leave the source in a valid, empty state
        other.size = 0;
    }

    ~Buffer() {
        delete[] data; // safe even if data is nullptr
    }
};

```

Note: *This is precisely the mechanism that makes `unique_ptr`'s move-only behaviour possible (Section 8.4.1): '`unique_ptr<int> p2 = move(p);`' invokes a move constructor that transfers ownership of the underlying pointer to `p2` and leaves `p` empty, rather than attempting (and failing) to copy it.*

15.3 Structured Bindings (C++17)

Structured bindings: syntax that lets you unpack a pair, tuple, struct, or array directly into several named variables in a single declaration, rather than accessing members one at a time (e.g. via `.first/.second` or `get<0>()`).

```

#include <map>
using namespace std;

int main() {
    map<string, int> ages = {{"Ayan", 22}, {"Riya", 24}};
}

```

```

for (const auto &[name, age] : ages) {    // unpacks each pair directly
    cout << name << " is " << age << endl;
}
return 0;
}

```

15.4 nullptr, override, and final

Keyword	Introduced	Purpose
nullptr	C++11	Type-safe null pointer literal (see Section 7.2).
override	C++11	Marks a function as overriding a base class virtual function; compiler-checked (Section 11.3).
final	C++11	Prevents a class from being further inherited from, or a virtual function from being further overridden.

```

class Base {
public:
    virtual void show() final { } // cannot be overridden by any further derived class
};
class Sealed final : public Base { }; // cannot be inherited from at all

```

15.5 constexpr (recap and expansion)

As introduced in Section 2.3, constexpr marks values (and, since C++11, functions) that can be evaluated at compile time. A constexpr function can run either at compile time or runtime depending on context, but is guaranteed to be a candidate for compile-time evaluation when its inputs are known at compile time.

```

constexpr int square(int x) {
    return x * x;
}

int main() {
    constexpr int result = square(5); // computed at COMPILE TIME, not runtime
    int arr[result];                // valid — result is a true compile-time constant
}

```

15.6 Smart Pointers, Lambdas, and Range-based for (cross-reference)

Three of the most impactful modern C++ features were already covered in detail where they naturally fit: smart pointers in Section 8.4, lambda expressions in Section 5.7, and the range-based for loop in Section 4.2.2. Together with auto, move semantics, and the STL, these features define what is generally meant by 'modern C++' as distinct from the closer-to-C subset of the language.

16. File Handling

C++ replaces C's FILE*-based functions (fopen, fprintf, fscanf, fclose) with stream classes from <fstream>, which integrate naturally with cin/cout-style I/O and handle their own cleanup via RAII.

16.1 File Stream Classes

Class	Purpose
-------	---------

Class	Purpose
ofstream	Output file stream — for WRITING to files.
ifstream	Input file stream — for READING from files.
fstream	Combined stream — supports both reading AND writing.

16.2 Writing to a File

```
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("data.txt"); // opens (creates/overwrites) the file

    if (outFile.is_open()) {
        outFile << "Name: Ayan" << endl;
        outFile << "Score: 95" << endl;
        outFile.close(); // good practice, though the destructor would close it too
    } else {
        cout << "Could not open file." << endl;
    }
    return 0;
}
```

Note: Because *ofstream* is an *RAII* type (Section 8.4, Section 9.5), the file is automatically closed when the *ofstream* object goes out of scope, even without an explicit *.close()* call. Calling *close()* explicitly is still good practice for clarity and for releasing the file handle before further code that needs it.

16.3 Reading from a File

```
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream inFile("data.txt");
    string line;

    if (inFile.is_open()) {
        while (getline(inFile, line)) { // reads one line at a time until EOF
            cout << line << endl;
        }
        inFile.close();
    } else {
        cout << "Could not open file." << endl;
    }
    return 0;
}
```

16.4 Reading Formatted Data with >>

```
ifstream inFile("scores.txt");
string name;
int score;

while (inFile >> name >> score) { // reads alternating word/number pairs
    cout << name << ": " << score << endl;
}
```

```
}
```

16.5 Append Mode and Other File Modes

Mode Flag	Effect
<code>ios::in</code>	Open for reading (default for <code>ifstream</code>).
<code>ios::out</code>	Open for writing (default for <code>ofstream</code>); truncates existing content.
<code>ios::app</code>	Open for writing, appending all output to the END of the file.
<code>ios::ate</code>	Open and immediately seek to the end of the file.
<code>ios::binary</code>	Open in binary mode (no text translation of line endings).

```
ofstream logFile("log.txt", ios::app); // append mode — preserves existing content
logFile << "New log entry" << endl;
```

16.6 Binary File I/O

As in C, binary mode reads/writes raw bytes rather than formatted text, useful for structured data like records or images.

```
struct Record { int id; double value; };

Record r{1, 99.5};
ofstream out("records.bin", ios::binary);
out.write(reinterpret_cast<char*>(&r), sizeof(Record));
out.close();

Record r2;
ifstream in("records.bin", ios::binary);
in.read(reinterpret_cast<char*>(&r2), sizeof(Record));
```

17. Preprocessor Directives and Multi-File Programs

The preprocessor works identically to C, since C++ inherits the C preprocessor wholesale.

17.1 Common Preprocessor Directives

Directive	Purpose
<code>#include</code>	Inserts the contents of another file (a header) before compilation.
<code>#define</code>	Defines a macro — a simple text substitution performed before compilation.
<code>#ifdef</code> / <code>#ifndef</code> / <code>#endif</code>	Conditional compilation — includes/excludes code based on whether a macro is defined.
<code>#pragma once</code>	A widely-supported (though non-standard) modern alternative to include guards.

17.2 Header Guards

Just as in C, header files need protection against being included multiple times in the same translation unit, which would cause duplicate-definition errors.

```
// student.h
#ifndef STUDENT_H    // traditional include guard (identical to C)
#define STUDENT_H

class Student {
public:
    void display();
};

#endif

// OR, the simpler modern alternative supported by virtually all compilers:
#pragma once
```

17.3 Multi-File C++ Programs

The header (.h)/source (.cpp) separation pattern from C carries over directly: declare a class's interface in a header, implement its member functions in a corresponding .cpp file.

```
// student.h — declarations only
#pragma once
#include <string>

class Student {
private:
    std::string name;
    int roll;
public:
    Student(std::string n, int r);
    void display();
};

// student.cpp — definitions, using the scope resolution operator ::
#include "student.h"
#include <iostream>
using namespace std;

Student::Student(string n, int r) : name(n), roll(r) {}

void Student::display() {
    cout << name << " - " << roll << endl;
}

// main.cpp — uses the class
#include "student.h"

int main() {
    Student s("Ayan", 101);
    s.display();
    return 0;
}

g++ main.cpp student.cpp -o program    // compile multiple .cpp files together
```

Note: `ClassName::functionName()` (the scope resolution operator from Section 3.4) is how a member function's definition, written outside the class body, is tied back to its class declaration — essential for keeping class interfaces in headers separate from their implementations.

18. Best Practices and Common Pitfalls

18.1 General Best Practices

- Prefer `std::string` over C-style char arrays, and `std::vector` over raw arrays, for safety and convenience (Sections 6.2, 14.1.1).
- Prefer smart pointers (`unique_ptr`, `shared_ptr`) over raw `new/delete` for owning dynamically allocated memory (Section 8.4).
- Pass large objects by const reference (`const T&`) rather than by value, to avoid unnecessary copying (Section 6.4.3).
- Always give a base class a virtual destructor if it has any virtual functions (Section 11.4).
- Use `override` when overriding virtual functions, so the compiler catches signature mismatches (Section 11.3).
- Initialize variables at the point of declaration, and prefer brace initialization `{}` to catch narrowing conversions (Section 2.1).
- Use `const` wherever a value or reference should not be modified — it documents intent and lets the compiler catch mistakes.
- Avoid `'using namespace std;'` in header files, since it pollutes the namespace of every file that includes that header.
- Use RAII (smart pointers, file streams, and custom destructors) instead of manual cleanup wherever possible (Sections 8.4, 9.5, 16.2).

18.2 Common Pitfalls

Pitfall	Why It's a Problem	Fix
Mismatching <code>new/delete</code> with <code>new[]/delete[]</code>	Undefined behaviour — may corrupt the heap or leak memory.	Always pair <code>new</code> with <code>delete</code> , and <code>new[]</code> with <code>delete[]</code> (Section 8.2).
Slicing — assigning a derived object to a base object (not pointer/reference)	Loses all derived-class data; only the base portion is copied.	Use pointers or references to base classes for polymorphic behaviour.
Forgetting a virtual destructor in a polymorphic base class	Derived class's destructor never runs when deleted via a base pointer — resource leak.	Always declare base class destructors virtual (Section 11.4).
Returning a reference to a local variable	The local variable is destroyed when the function returns, leaving a dangling reference.	Return by value, or use a pointer to dynamically allocated/longer-lived memory instead.
Comparing C-style strings with <code>==</code>	Compares the pointers (addresses), not the actual text content.	Use <code>std::string</code> and its <code>==</code> operator, or <code>strcmp()</code> for char arrays (Section 6.2).
Off-by-one access on vector/array	Same undefined behaviour as in C — no automatic bounds checking with operator <code>[]</code> .	Use <code>.at(i)</code> when bounds-checking matters, or carefully verify loop bounds.

18.3 C++ vs C — Summary Table

Feature	C	C++
Paradigm	Procedural	Procedural, OOP, and generic
Memory management	malloc/free	new/delete, and smart pointers
String handling	char arrays + string.h functions	std::string class
Data structures	Manually implemented (Section 13 of C notes)	Built-in via the STL (Section 14)
Error handling	Return codes / errno	Exceptions (try/throw/catch)
Code reuse	Function libraries only	Inheritance, templates, operator overloading
Type safety	Looser (e.g. void* casting)	Stricter (templates, typed casts, references)

18.4 Where to Go Next

This document covers the core of the language as commonly tested and used in interviews, coursework, and everyday application development. Deeper topics worth exploring afterward include: move semantics in more depth and perfect forwarding, the Rule of Three/Five/Zero for resource-managing classes, multithreading with `std::thread` (`<thread>`), the ranges library (C++20), and concepts/coroutines (C++20). The official cppreference.com remains the most reliable reference for exact language and library behaviour across standard versions.